INTEGRIDAD, TRIGGERS, PROCEDURES Y AUTORIZACIÓN

Clase 4 · Diseño de Base de Datos

Ingeniería en Informática · 2.º año · Universidad Católica de Córdoba

Agenda

0. **Recordatorio — de datos a reglas**
1. **Restricciones de integridad — `CHECK`, `CASCADE` y compañía**
2. **Triggers — qué son, cómo se usan, cuándo conviene**
3. **Stored procedures y funciones — concepto e invocación**
4. **Autorización — `GRANT`, `REVOKE`, roles, vistas como mecanismo de seguridad**

Cierre del módulo SQL. Después de esta clase, la base no solo guarda datos: aplica reglas, automatiza tareas y decide quién puede ver qué.

De datos a reglas

En las tres clases anteriores aprendimos a **consultar** y **modificar** datos. Hoy le agregamos a la base la capacidad de:

RESTRINGIR

Definir reglas que los datos **deben cumplir**. Si alguien intenta violarlas, la operación se rechaza.

REACCIONAR

Ejecutar lógica **automáticamente** cuando ocurren eventos (triggers) o cuando se la invoca explícitamente (procedures).

AUTORIZAR

Definir **quién puede hacer qué** sobre cada tabla, columna o vista.

✓ IDEA FUERZA

Las reglas dentro de la base son **independientes de la aplicación**: aunque cambie el front, aunque haya múltiples programas accediendo, las reglas se cumplen siempre.

01

Restricciones de integridad

Reglas que los datos siempre tienen que cumplir

Las que ya conocemos

Vienen desde la Clase 1, cuando creamos las tablas de SiGUE.

Restricción	Qué garantiza
`PRIMARY KEY`	Cada fila se identifica de forma única. Implica `NOT NULL` + `UNIQUE`.
`FOREIGN KEY`	Los valores de la FK existen como PK en la tabla referenciada (integridad referencial).
`NOT NULL`	La columna nunca puede quedar vacía.
`UNIQUE`	No se repiten valores. *Sutileza*: dos `NULL` no se consideran duplicados.
`DEFAULT`	Si no se especifica valor al insertar, se usa el predeterminado.

HOY AGREGAMOS

`CHECK` para reglas sobre valores, y las **acciones referenciales** (`ON DELETE` / `ON UPDATE`) que decidieron qué hacer cuando se borra o modifica una fila referenciada.

CHECK: reglas sobre valores

Un predicado que debe cumplirse para cada fila. Si no se cumple, la operación es rechazada.

```sql
-- Al crear la tabla
CREATE TABLE notas (
    id_nota   INT NOT NULL AUTO_INCREMENT,
    id_alumno INT NOT NULL,
    nota      DECIMAL(4,2) CHECK (nota BETWEEN 1 AND 10),
    aprobado  TINYINT(1),
    PRIMARY KEY (id_nota),
    CONSTRAINT chk_coherencia CHECK (aprobado = 1 OR nota < 4)
);

-- O agregar a posteriori
ALTER TABLE departamentos
ADD CONSTRAINT chk_presupuesto_positivo CHECK (presupuesto > 0);
```

⚠ MYSQL — VERSIÓN

MySQL **solo aplica `CHECK` desde la versión 8.0.16**. En versiones anteriores, la cláusula se parseaba pero se ignoraba silenciosamente. Si estás en una instalación vieja, el `CHECK` no protege nada.

La pregunta de fondo

Una asignatura tiene FK desde varias comisiones. **¿Qué pasa si se borra esa asignatura?**

LAS COMISIONES QUEDAN "HUÉRFANAS"

Conservan un `id_asignatura` que ya no existe → se rompe la integridad referencial.

SQL permite definir *de antemano* qué hacer en estos casos, con dos cláusulas en la FK:

```
-- Plantilla: definir qué hacer si la fila referenciada se borra o se modifica
CREATE TABLE comisiones (
    ...
    FOREIGN KEY (id_asignatura) REFERENCES asignaturas(id_asignatura)
        ON DELETE <acción>
        ON UPDATE <acción>
);
```

`ON DELETE` se dispara cuando se borra la fila referenciada; `ON UPDATE` cuando se modifica la PK referenciada. Cada uno acepta cinco acciones posibles.

Las cinco acciones posibles

Acción	Qué hace cuando se borra/actualiza la fila referenciada
`NO ACTION`	**Rechaza** la operación. Es el comportamiento por defecto si no se declara nada. El chequeo es al final de la sentencia.
`RESTRICT`	Similar a `NO ACTION`, pero el chequeo es **inmediato**. En MySQL es el efectivo por defecto.
`CASCADE`	Propaga el cambio: borra o actualiza también las filas hijas.
`SET NULL`	Pone `NULL` en la columna FK. Requiere que la columna admita `NULL`.
`SET DEFAULT`	Pone el valor por defecto del dominio. Soporte limitado en MySQL.

✓ TIP

Las cascadas pueden encadenarse entre tablas. Si una cascada provoca una violación que no se puede resolver, **se aborta toda la transacción**.

Cuándo usar cada acción — caso SiGUE

"Borrar una asignatura" tiene efectos distintos según la decisión de diseño:

Si elijo…	Lo que pasa	Para qué sirve
`ON DELETE CASCADE`	Al borrar una asignatura, sus comisiones se borran. Y si `inscripcion_alumno_comision` también tiene CASCADE, las inscripciones también.	Limpieza completa. Útil cuando los datos hijos **no tienen sentido** sin el padre.
`ON DELETE RESTRICT`	No deja borrar la asignatura si tiene comisiones asociadas. Hay que limpiarlas primero.	Protección. Útil cuando borrar la fila padre debe ser una **decisión consciente**.
`ON DELETE SET NULL`	La asignatura desaparece y las comisiones quedan con `id_asignatura = NULL`.	Conservar el historial. Rara vez tiene sentido para FK obligatorias.

REGLA PRÁCTICA PARA SIGUE

- **Datos académicos sensibles** (notas, exámenes, inscripciones aprobadas) → `RESTRICT`. No queremos perder historial por error.
- **Vínculos efímeros** (asistencias diarias, inscripciones provisorias) → `CASCADE` es razonable.
- **Casi nunca** `SET NULL` en datos académicos: dejaría huérfanos sin contexto.

02

Triggers

Código que se ejecuta automáticamente

¿Qué es un trigger?

DEFINICIÓN

Un **trigger** ("disparador") es un bloque de código **asociado a una tabla** que el motor ejecuta **automáticamente** cuando ocurre un evento sobre ella: una inserción, una actualización o un borrado.

Nadie lo invoca explícitamente: el motor lo dispara cuando se cumple la condición. Sirve para:

VALIDACIONES COMPLEJAS

Reglas que `CHECK` no puede expresar (por ejemplo, validar contra otra tabla, comparar con valores previos, contar filas relacionadas).

LÓGICA AUTOMÁTICA

Mantener auditoría (log), calcular columnas derivadas, sincronizar con otra tabla, notificar.

Sintaxis MySQL básica

```
-- Plantilla genérica de un trigger
DELIMITER //

CREATE TRIGGER nombre_del_trigger
    {BEFORE | AFTER}        -- antes o después del evento
    {INSERT | UPDATE | DELETE}  -- el evento que dispara
    ON nombre_tabla
    FOR EACH ROW
BEGIN
    -- código a ejecutar
```

Palabra reservada	Qué significa
`DELIMITER //`	Cambia temporalmente el separador de sentencias para que los `;` internos del trigger no terminen la sentencia. Al final se vuelve a `;`.
`BEFORE` / `AFTER`	Cuándo se ejecuta el código: antes o después del evento (`BEFORE` permite modificar la fila entrante).
`FOR EACH ROW`	El código se ejecuta **una vez por cada fila afectada** por la sentencia.
`BEGIN ... END`	Delimitan el bloque de instrucciones del trigger (igual que llaves en otros lenguajes).
`NEW` / `OLD`	`NEW` = fila entrante (`INSERT`, `UPDATE`); `OLD` = fila previa (`UPDATE`, `DELETE`). Se accede a columnas con `NEW.columna`.

Ejemplo SiGUE: calcular `aprobado` al insertar

Cada vez que se inserta una nota, queremos que la columna `aprobado` se complete sola (1 si la nota ≥ 4, 0 si no).

```sql
-- Marca aprobado=1 cuando la nota es ⩾ 4, sin que la app tenga que calcularlo
DELIMITER //

CREATE TRIGGER tr_calc_aprobado
BEFORE INSERT ON notas
FOR EACH ROW
BEGIN
    IF NEW.nota ⩾ 4 THEN
        SET NEW.aprobado = 1;
    ELSE
        SET NEW.aprobado = 0;
    END IF;
END//

DELIMITER ;
```

Palabra reservada	Qué significa
`IF ... THEN ... ELSE ... END IF`	Estructura condicional dentro de stored programs. Cierra con `END IF` obligatorio.
`SET` (en triggers)	Asigna valor a una variable o a una columna de `NEW`. No confundir con el `SET` de `UPDATE`.

Trigger: ¿cuándo conviene?

CONVIENE CUANDO…

- La regla es **esencial** y debe cumplirse aunque la modificación venga de otra app.
- `CHECK` no alcanza: hay que consultar otra tabla, comparar con el valor previo, contar registros.
- Querés **auditoría** automática (log de cambios).
- Necesitás **denormalizar** de forma segura (ej.: mantener un contador o un valor agregado).

MEJOR EVITAR CUANDO…

- La regla puede vivir en la aplicación y solo una app accede a la BD.
- Es una regla simple de valor: usá `CHECK`.
- El trigger encadena lógica compleja con varios saltos: se vuelve **difícil de seguir** ("magia oculta").
- Puede causar **recursión** (trigger que modifica una tabla que dispara otro trigger).

⚠ COSTO PRINCIPAL

Los triggers son invisibles desde afuera: alguien hace un `INSERT` simple y se ejecuta código del que no tiene noticia. Documentar y mantenerlos bajo control de versiones es **imprescindible**.

03

Procedures y funciones

Lógica almacenada en la base, invocable

Stored procedure: una rutina con nombre

Un **procedure** es código almacenado en la base, con un nombre, que se invoca explícitamente con `CALL`. Puede recibir parámetros y devolver resultados.

```sql
-- Inscribe un alumno a una comisión, validando que la comisión exista
DELIMITER //

CREATE PROCEDURE inscribir_alumno (
    IN p_id_alumno   INT,
    IN p_id_comision INT
)
BEGIN
    IF EXISTS (SELECT 1 FROM comisiones WHERE id_comision = p_id_comision) THEN
        INSERT INTO inscripcion_alumno_comision
            (id_alumno, id_comision, fecha_inscripcion, estado)
        VALUES (p_id_alumno, p_id_comision, CURDATE(), 'activa');
```

Palabra reservada	Qué significa
`IN` / `OUT` / `INOUT`	Tipo de parámetro. `IN` = entrada (default), `OUT` = salida (el procedure le asigna valor), `INOUT` = ambos.
`IF EXISTS (subquery)`	Predicado booleano: TRUE si la subconsulta produce al menos una fila. Mismo concepto que el `EXISTS` de Clase 3.
`CALL`	Sentencia que invoca un procedure por su nombre.

Funciones (UDF): la prima de los procedures

Característica	Stored procedure	Función (UDF)
Devuelve…	Cero, uno o varios result sets.	Exactamente **un valor**.
Se invoca con…	`CALL` (sentencia propia).	Dentro de un `SELECT`, `WHERE`, etc., como cualquier función nativa.

```sql
-- Devuelve el promedio de notas aprobadas
-- del alumno; 0 si no tiene ninguna
DELIMITER //
CREATE FUNCTION promedio_alumno (p_id INT)
RETURNS DECIMAL(4,2)
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE prom DECIMAL(4,2);
    SELECT AVG(nota) INTO prom
    FROM notas
    WHERE id_alumno = p_id AND aprobado = 1;
    RETURN IFNULL(prom, 0);
END//
DELIMITER ;

-- Uso como columna calculada
SELECT apellido, promedio_alumno(id_alumno)
FROM alumnos;
```

Palabra	Significado
`RETURNS` *tipo*	Declara el tipo del valor que devuelve la función.
`DETERMINISTIC`	Promesa: con los mismos inputs siempre devuelve el mismo output. Ayuda al motor a optimizar.
`READS SQL DATA`	Declara que la función *lee* tablas pero no las modifica.
`DECLARE`	Crea una variable local dentro del cuerpo.
`SELECT ... INTO`	Toma el resultado de un `SELECT` que devuelve una sola fila y lo asigna a variables.
`RETURN`	Devuelve un valor y termina la ejecución.
`IFNULL(x, y)`	Devuelve **x** si no es NULL; si lo es, devuelve **y**.

Procedures y funciones: ¿cuándo conviene?

CONVIENE CUANDO…

- La lógica involucra **muchas consultas** y conviene resolverla cerca de los datos.
- Varias aplicaciones tienen que aplicar **la misma lógica de negocio**: tenerla una sola vez en la base evita duplicaciones.
- Hay **seguridad** de por medio: el procedure puede acceder a tablas restringidas con sus propios privilegios, sin dárselos al usuario.
- Performance: menos viajes app ↔ base.

MEJOR EVITAR CUANDO…

- Una sola app accede a la BD y la lógica se entiende mejor en el código de la app.
- El lenguaje del procedure (MySQL: SQL/PSM) es **menos productivo** que el lenguaje de tu app para esa tarea.
- No está claro cómo **versionar y testear** los procedures (sin tooling cuesta).
- Hay que migrar a otro motor: el código no es portable.

REGLA PRÁCTICA

Lógica de **integridad y negocio crítico** que debe valer "siempre": triggers + CHECK + procedures. **Lógica de presentación o de flujo de la app**: en la app.

04

Autorización

Quién puede hacer qué

Privilegios sobre los datos

Cada operación SQL tiene su privilegio. Quien crea una tabla recibe automáticamente todos los privilegios sobre ella.

`SELECT`	`INSERT`	`UPDATE`	`DELETE`	`REFERENCES`
Leer filas	Crear filas	Modificar filas	Borrar filas	Crear FK contra la tabla

GRANULARIDAD

Los privilegios se aplican **por tabla o por columnas específicas**. `UPDATE` e `INSERT` aceptan lista de columnas: se puede dar permiso para actualizar solo una columna concreta.

*Para "permiso por fila" no hay sintaxis nativa: se emula con **vistas filtradas** (lo vemos en dos slides).*

GRANT y REVOKE

```
-- Sintaxis general
GRANT  <privs> ON <tabla|vista> TO   <usuarios>;
REVOKE <privs> ON <tabla|vista> FROM <usuarios>;

-- Lectura de toda la base para bedelía
GRANT SELECT ON sigue.* TO 'bedelia'@'localhost';

-- Modificar solo la columna "presupuesto"
GRANT UPDATE (presupuesto) ON sigue.departamentos
TO 'jefe_finanzas'@'localhost';

-- Control total sobre la tabla notas
GRANT ALL PRIVILEGES ON sigue.notas
TO 'rector'@'localhost';

-- Quitar permisos de escritura
REVOKE INSERT, UPDATE, DELETE ON sigue.alumnos
FROM 'bedelia'@'localhost';

-- Privilegio para TODOS los usuarios
GRANT SELECT ON sigue.asignaturas TO PUBLIC;
```

Sintaxis	Qué significa
`GRANT`	Otorga privilegios. `TO` indica a quién.
`REVOKE`	Quita privilegios. `FROM` indica a quién.
`ON tabla.*`	Todas las tablas de la base `tabla`.
`(col1, col2)`	Limita el privilegio a columnas específicas (solo `UPDATE` e `INSERT`).
`ALL PRIVILEGES`	Atajo: todos los privilegios disponibles sobre el objeto.
`'user'@'host'`	Identidad MySQL: usuario + máquina desde la que se conecta.
`PUBLIC`	Otorga a todos los usuarios actuales y futuros.

Roles: agrupar privilegios

En vez de repetir los mismos `GRANT` a cada profesor nuevo, se define un **rol** con esos privilegios y se le otorga el rol a cada usuario.

```sql
-- Definimos un rol que agrupa los permisos del profesor titular
CREATE ROLE 'profesor_titular';

-- Le asignamos privilegios al rol (no a un usuario concreto)
GRANT SELECT, INSERT, UPDATE ON sigue.notas      TO 'profesor_titular';
GRANT SELECT ON                      sigue.comisiones TO 'profesor_titular';

-- Le damos el rol a un usuario concreto (hereda todos sus privilegios)
GRANT 'profesor_titular' TO 'martinez'@'localhost';
```

VENTAJAS

- Un nuevo profesor → un único `GRANT` al rol.
- Cambiar permisos del rol cambia los de todos los usuarios automáticamente.
- Los roles se pueden **anidar** (un rol "decano" hereda de "profesor_titular").

Sintaxis	Qué significa
`CREATE ROLE`	Crea un rol vacío (un "contenedor" de privilegios).
`GRANT` *privs* `TO` *rol*	Asigna privilegios al rol (igual sintaxis que a un usuario).
`GRANT` *rol* `TO` *usuario*	Le da el rol a un usuario — hereda todos sus privilegios.

Vistas como mecanismo de seguridad

SQL no tiene "permiso por fila" nativo, pero se puede emular con una **vista filtrada**: se le da permiso al usuario sobre la vista, no sobre la tabla base.

```sql
-- Vista: cada docente solo ve SUS comisiones (filtrado por departamento)
CREATE VIEW comisiones_dpto_informatica AS
SELECT c.*
FROM comisiones c
INNER JOIN docentes d ON c.id_docente = d.id_docente
WHERE d.id_departamento = 3;

-- Damos permiso solo sobre la vista
GRANT SELECT ON sigue.comisiones_dpto_informatica
```

TAMBIÉN FUNCIONA PARA COLUMNAS SENSIBLES

Una vista que no expone la columna `salario_base` permite que personal administrativo consulte la información de docentes sin ver los sueldos.

⚠ DETALLE DE SEGURIDAD

El creador de la vista **no recibe** privilegios extra: solo puede crear vistas sobre tablas a las que ya tiene acceso. Si no, el `CREATE VIEW` falla.

Pausa: probalo vos

Consigna: implementá dos reglas de negocio sobre SiGUE.

1. La nota debe estar entre 1 y 10 (no se acepta cargar valores fuera de ese rango).

```
ALTER TABLE notas
ADD CONSTRAINT chk_nota_rango
CHECK (nota BETWEEN 1 AND 10);
```

2. Si una asignatura se borra, sus comisiones se borran automáticamente — pero **no** sus inscripciones (esas requieren confirmación manual).

```
ALTER TABLE comisiones
ADD CONSTRAINT fk_com_asig
FOREIGN KEY (id_asignatura) REFERENCES asignaturas(id_asignatura)
ON DELETE CASCADE;

ALTER TABLE inscripcion_alumno_comision
ADD CONSTRAINT fk_inscr_com
```

EFECTO COMBINADO

Al borrar una asignatura, MySQL intenta borrar sus comisiones (CASCADE). Si alguna de esas comisiones tiene inscripciones, el RESTRICT de la otra FK **aborta toda la operación** — se preserva el historial.

Lo que vimos hoy · cierre del módulo SQL

- **Integridad**: `CHECK` + acciones referenciales (`CASCADE`, `RESTRICT`, `SET NULL`…).
- **Triggers**: código automático en eventos. `BEFORE/AFTER` + `NEW/OLD`.
- **Procedures**: rutinas con nombre invocadas con `CALL`.
- **Funciones**: devuelven un valor, se usan dentro de un `SELECT`.
- **Autorización**: `GRANT` / `REVOKE` sobre tablas, columnas o vistas.
- **Roles**: agrupar privilegios para reutilizarlos.
- **Vistas como seguridad**: filtrado por filas o por columnas.

RESUMEN DEL MÓDULO SQL

Clase 1: leer · Clase 2: resumir · Clase 3: combinar y modificar · Clase 4: gobernar.
A partir de acá: **relaciones avanzadas**, álgebra relacional, transacciones, indexación y procesamiento de consultas (para el examen final).

Práctico de cierre del módulo: caso integrador sobre SiGUE — modelado, consultas, vistas, triggers y permisos.